

# DNA

## Disease Diagnosis Using Random Forest

### HCV laboratory classification study

Team DNA | College project report | 6 October 2026

#### Abstract

DNA is a working educational application that uses a Random Forest to classify laboratory records into five categories from the official UCI HCV dataset. The experiment uses 492 training records and 123 held-out test records. The evaluated model achieves 95.12% accuracy and a macro F1 of 0.612. The difference between these metrics reflects a substantial class imbalance and weaker performance on rare categories.

The application combines laboratory input, five class scores, missing-value disclosure, and an Explain and Explore feature. This feature shows how replacing one input with its training median changes the predicted class score and allows comparison of edited input panels. The report covers the method, implementation, measured results, limitations, and a path to adding separate disease modules.

#### Contents

| Section                            | Page |
|------------------------------------|------|
| Problem and dataset                | 2    |
| Methodology                        | 3    |
| Implementation and user experience | 4    |
| Results and evaluation             | 5    |
| Novelty feature                    | 6    |
| Limitations and future expansion   | 7    |
| Conclusion and references          | 8    |

#### Educational scope

DNA predicts research-dataset categories. It does not confirm a diagnosis, recommend treatment, or establish that a blood donor is disease-free.

# Problem and dataset

## Problem statement

Build a disease classification model using Random Forest, connect it to an understandable interface, and evaluate how well it distinguishes diagnosis categories from laboratory results. The first module focuses on HCV-related data; broader disease coverage is planned as separate modules after this study is understood.

## Dataset selection

The official UCI HCV dataset contains laboratory and demographic records, with five supplied categories: blood donor, suspected blood donor, hepatitis, fibrosis, and cirrhosis [1]. The local file contains 615 records and 12 predictors. The patient identifier and Category column are excluded from the model inputs.

| Category              | Records |
|-----------------------|---------|
| Blood donor           | 533     |
| Suspected blood donor | 7       |
| Hepatitis             | 24      |
| Fibrosis              | 21      |
| Cirrhosis             | 30      |

## Inputs

The predictors are age, recorded sex, and ten laboratory fields: ALB, ALP, ALT, AST, BIL, CHE, CHOL, CREA, GGT and PROT. The interface displays their expanded names. UCI does not provide laboratory units in the published variable table [1], so DNA uses original dataset-scale values and supplies research examples. Clinical unit conversions or normal-range flags are not inferred.

## Data quality

Inspection of the downloaded file found missing values in ALB (1), ALP (18), ALT (1), CHOL (10), and PROT (1). Other input columns have no missing entries. The source archive and original CSV are retained. The training outputs record a SHA-256 checksum, class counts, missing counts and row identifiers for reproducibility.

## Objectives

The project demonstrates the complete workflow and lets users inspect input effects. It retains uncommon categories and reports their limitations alongside overall performance.

# Methodology

## Random Forest

A Random Forest combines multiple decision trees trained with randomness. Bootstrap samples vary the records seen by each tree, while random feature subsets vary the split candidates. Aggregation reduces dependence on any one tree [2]. The scikit-learn implementation averages class probabilities across trees and selects the highest-scoring class [3].

| Configuration               | Value                                    |
|-----------------------------|------------------------------------------|
| Trees                       | 400                                      |
| Minimum samples per leaf    | 2                                        |
| Feature selection at splits | Square root of transformed feature count |
| Class weights               | balanced_subsample                       |
| Random seed                 | 42                                       |
| Tree depth                  | Unrestricted                             |

## Experiment sequence

1. Load the official CSV and map its five target strings to fixed class identifiers.
2. Make a stratified 80/20 split into 492 training and 123 test records.
3. Run five-fold stratified cross-validation within the training partition.
4. Fit the same pipeline on the complete training partition.
5. Evaluate once on the test records and save that exact model.

## Preprocessing and leakage control

A ColumnTransformer imputes numerical inputs with training medians and one-hot encodes recorded sex. This transformer is inside the pipeline passed to cross-validation, so each fold learns its own medians and category encoding. Test records do not influence preprocessing or fitting. The training script also checks for duplicate feature rows crossing the train/test boundary.

## Evaluation design

The main measures are accuracy, macro F1, balanced accuracy, and per-class precision and recall. Macro F1 gives equal weight to each category. Balanced accuracy averages recall across categories. A most-frequent-class baseline measures how far a classifier can get by always predicting the dominant class. Settings were fixed before evaluation; no test-set tuning or hyperparameter search is performed.

# Implementation and user experience

## Application architecture

Training: source CSV to split to preprocessing and Random Forest to model artifacts and evaluation outputs. Inference: laboratory form to validated JSON to the saved pipeline to category scores and explanations. The Flask server runs on the local loopback address.

| Component            | Responsibility                                                              |
|----------------------|-----------------------------------------------------------------------------|
| modules.py           | Defines the HCV input contract and independent artifact location.           |
| train.py             | Trains the pipeline, evaluates it and exports evidence.                     |
| app.py               | Validates inputs, loads the model and computes predictions and sensitivity. |
| templates and static | DNA branding, responsive layout and client-side interactions.               |
| artifacts/hcv        | Model, metrics, examples, predictions and charts.                           |
| test_model.py        | Checks data separation, validation, prediction consistency and sensitivity. |

## DNA interface

The Lab Explorer separates input, prediction and explanation. Users can load a research sample or enter a panel. Results show every class score, not just the winning label. The evaluation section displays macro F1 alongside accuracy. The layout adapts to narrow screens, labels inputs clearly, and clears stale results when values change.

## Input policy

Age and recorded sex are required. Numerical values must be finite and non-negative, with a broad technical upper bound to avoid numeric overflow. Up to three missing lab values can use training medians; four or more are rejected. This is a demonstration policy, not a validated clinical rule. The output lists all substituted values and flags values beyond observed training ranges.

## Run the working model

From PowerShell in the project folder, run:

```
.\.venv\Scripts\python.exe app.py
```

Then visit <http://127.0.0.1:5000> and choose Load sample followed by Analyze test panel. Pin the result, edit a lab value, and analyze again to compare. The README includes fresh-install and retraining commands.

# Results and evaluation

| Metric            | Random Forest | Majority baseline |
|-------------------|---------------|-------------------|
| Accuracy          | 95.12%        | 86.99%            |
| Macro F1          | 0.612         | 0.186             |
| Balanced accuracy | 58.67%        | 20.00%            |

| Class                 | Precision | Recall  | Test count |
|-----------------------|-----------|---------|------------|
| Blood donor           | 97.27%    | 100.00% | 107        |
| Suspected blood donor | 0.00%     | 0.00%   | 1          |
| Hepatitis             | 75.00%    | 60.00%  | 5          |
| Fibrosis              | 50.00%    | 50.00%  | 4          |
| Cirrhosis             | 100.00%   | 83.33%  | 6          |

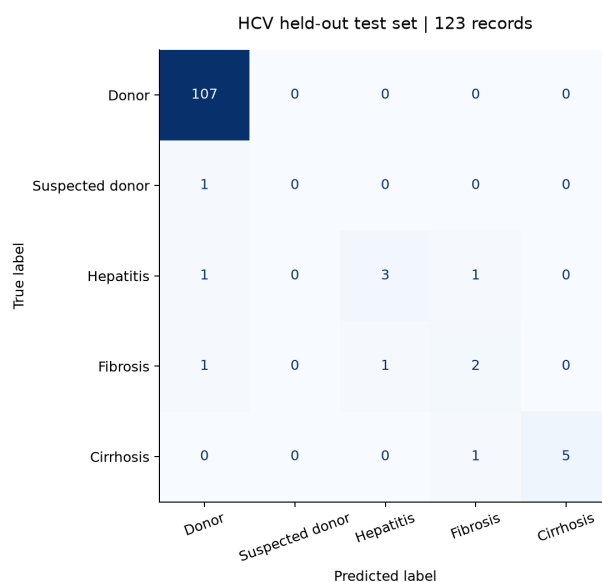


Figure 1. Actual categories appear in rows and predicted categories in columns.

The model correctly classifies 117 of 123 test records. However, the single suspected-donor test case is missed. Hepatitis recall is 60% on five cases and fibrosis recall is 50% on four cases. These very small groups make estimates unstable. A precision value of zero for a class with no predictions is the reporting convention used here, not a stable estimate of future precision.

Five-fold training validation: mean accuracy 92.27% and mean macro F1 0.587, with macro-F1 standard deviation 0.140. Fold variation is descriptive, not a deployment confidence interval.

# Novelty feature

## Explain and Explore

The project contribution is an integrated explanation and comparison workflow. After prediction, DNA ranks the observed lab inputs by how strongly replacing each one with its training median changes the current winning category score. This helps the examiner explore the actual fitted model rather than seeing an unexplained label.

## How the explanation is computed

First, the model scores the complete input panel. For each observed lab value, a copy of that panel replaces only that feature with its training median. The original winning category is held fixed when comparing the scores. The score difference is displayed in percentage points. A positive difference means the observed value supports that score relative to the selected reference. Missing fields are excluded from this explanation.

## Interactive comparison

The user can pin a result, change values, and analyze again. DNA reports the changed inputs, the new winning category and the change in the pinned category score. Comparisons use the same fitted model. Results can be exported as JSON, including inputs, class scores, imputation disclosures and sensitivity values.

## Concrete example

Load a held-out sample and run the model. Pin the output, alter ALT, and run again. The comparison panel shows whether the class changes and how much the original category score moves. A lack of movement is also informative: the ensemble may not cross relevant split thresholds for that change.

## What is novel in this project

The feature combines transparent missing-value handling, local input sensitivity and interactive comparison inside one student-facing application. It is a software contribution for this project, not a claim that one-at-a-time sensitivity analysis or Random Forest is a new research method.

## Interpretation limits

The explanation is not SHAP and does not provide additive feature attribution. Correlated tests can make one-at-a-time changes unrealistic, and medians are dataset references rather than healthy clinical targets. The effects cannot justify treatment recommendations. The top-score review flag at 65% is illustrative and has not been selected for clinical sensitivity or calibrated risk.

## Feature verification

Automated checks compare the displayed sensitivity calculation against direct pipeline inference on the same modified inputs. Separate tests check invalid input rejection, missing-value disclosures, and agreement between endpoint predictions and every saved test prediction.

# Limitations and future expansion

## Current limitations

The dataset is small and heavily dominated by blood donors. Rare categories are represented by very few records, limiting both training and evaluation. The model has no external hospital validation and no probability calibration. Random splitting cannot establish performance across different populations, instruments, or data collection procedures.

## A category is not a confirmed diagnosis

DNA learns the categories in the source file. A donor prediction does not rule out disease, and a disease-category prediction does not establish the clinical condition. The input-range warning checks individual values only; it cannot establish whether an entire panel is realistic or whether its units match the research data.

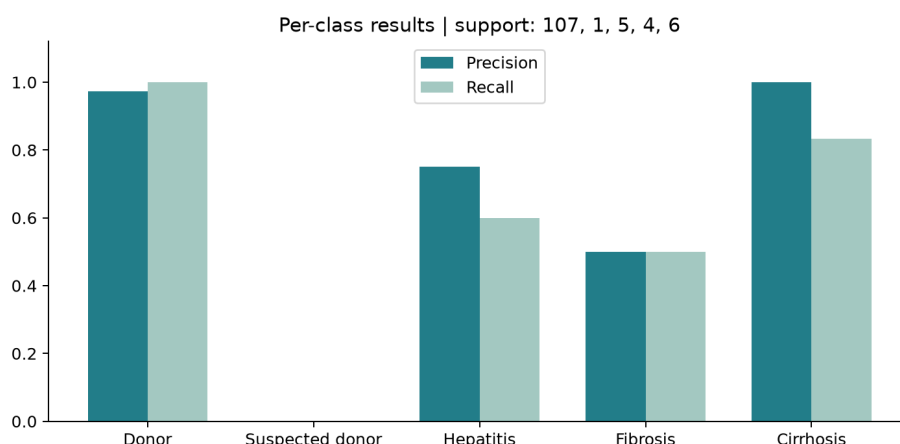


Figure 2. Per-class precision and recall expose weaknesses hidden by overall accuracy.

## Adding more diseases later

Keep each disease dataset and model separate. A new module needs a source citation and license, documented input units, explicit target definitions, preprocessing fitted within training folds, and its own test evaluation. Only then should it be exposed in the DNA interface. The module registry and namespaced HCV artifacts provide a starting structure; HCV is the only implemented training workflow today.

## Why unrelated datasets should not simply be joined

Kidney, thyroid and liver datasets have different tests and populations. Combining rows with zero-filled absent tests may reveal the data source rather than disease. Separate modules support independent evaluation and overlapping disease flags.

## Next study improvements

Before adding modules, examine HCV errors and calibration using training-only validation. Avoid tuning on the test set. Seek more representative external data.

# Conclusion and references

## Conclusion

DNA implements a working HCV laboratory classification system using Random Forest. The measured test accuracy is 95.12%, while macro F1 is 0.612 and balanced accuracy is 58.67%. Together these measures show that the system performs strongly on the dominant donor category but remains limited on rare categories.

The project demonstrates data preparation, leakage-free preprocessing, multiclass evaluation, saved-model inference and a branded interface. Explain and Explore makes model behavior inspectable without treating model responses as clinical rules. Future disease coverage should proceed through separately validated modules after the current workflow and its limitations are understood.

## References

[1] Lichtinghagen, R., Klawonn, F., and Hoffmann, G. (2020). HCV data. UCI Machine Learning Repository. CC BY 4.0.

<https://doi.org/10.24432/C5D612>

[2] Breiman, L. (2001). Random Forests. Machine Learning, 45, 5-32.

<https://doi.org/10.1023/A:1010933404324>

[3] scikit-learn. RandomForestClassifier API documentation.

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

[4] Hoffmann, G., Bietenbeck, A., Lichtinghagen, R., and Klawonn, F. (2018). Using machine learning techniques to generate laboratory diagnostic pathways - a case study. Journal of Laboratory and Precision Medicine.

<https://doi.org/10.21037/jlpm.2018.06.01>

## Reproduction evidence

All reported model results come from artifacts/hcv/metrics.json and test\_predictions.csv. The source data is data/hcv/hcvdat0.csv. The source authors and UCI retain the dataset attribution; Team DNA provides the project implementation.

The original dataset publication [4] provides background on laboratory diagnostic pathways. This report does not substitute its results for the evaluation of DNA.

Source file SHA-256: 1b62c8453b7098b695c549b19cfe070458ce6b909f0e990b6cfce7ab3c44c273